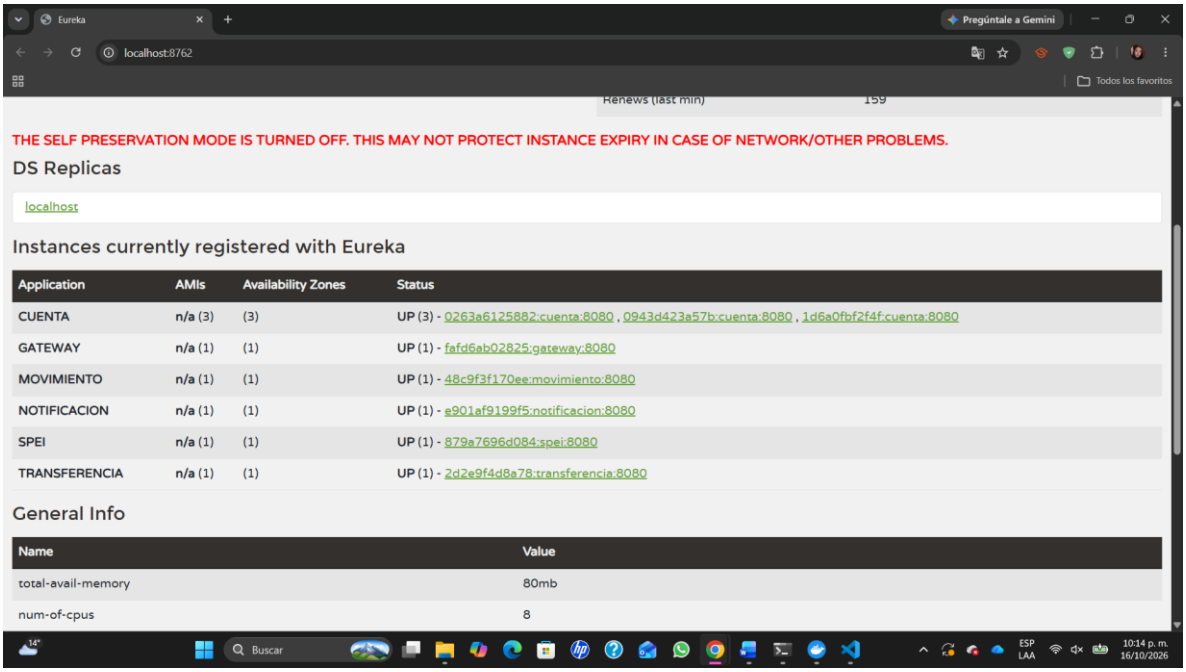
López Hernández Miriam Amisadai

Entorno Utilizado

Partí del proyecto usado en S07 y preparé una copia independiente para S08. Para evitar que los contenedores de S07 se mezclaran con los de S08, cambié el nombre del proyecto a mexi-banco-s08, la red a mexibanco-s08 y los puertos publicados del Gateway a 8081 y Eureka a 8762.



Parte 0: Reproducir el problema

Creé dos cuentas de prueba en la base de S08 y consulté sus saldos por el Gateway. La cuenta de origen tenía $1,000 y la de destino $500. Para que pudiera ver el problema puse una pausa de 15 segundos entre el cargo y el abono. Durante la transferencia de $100 detuve las tres réplicas de cuenta y después las volví a iniciar.

Consulte los saldos antes de hacer la falla

```
PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\sd-2027-1\entregas\lopez_miriam\s08\mexi-banco> Invoke-RestMethod -Uri "http://localhost:8081/api/cuentas/123456789012345678" -Method Get

id clabe              titular            saldo
-- -----              -------            -----
 1 123456789012345678 Prueba S08 Origen 1000.00


PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\sd-2027-1\entregas\lopez_miriam\s08\mexi-banco> Invoke-RestMethod -Uri "http://localhost:8081/api/cuentas/987654321098765432" -Method Get

id clabe              titular            saldo
-- -----              -------            -----
 2 987654321098765432 Prueba S08 Destino 500.00
```

La solicitud de transferencia me devolvió un error HTTP 500

```
PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\sd-2027-1\entregas\lopez_miriam\s08\m
exi-banco> Invoke-RestMethod -Uri "http://localhost:8081/api/transferencias" -Method Post -
ContentType "application/json" -Body $transferencia
Invoke-RestMethod : Error en el servidor remoto: (500) Error interno del servidor.
En línea: 1 Carácter: 1
+ Invoke-RestMethod -Uri "http://localhost:8081/api/transferencias" -Me ...
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest)
[Invoke-RestMethod], WebException
    + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.I
nvokeRestMethodCommand
```

Aquí se ven las tres réplicas de cuenta detenidas durante la prueba

```
PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\sd-2027-1\entregas\lop
ez_miriam\s08\mexi-banco> docker compose stop cuenta
[+] stop 3/3
 ✔Container mexi-banco-s08-cuenta-2 Stopped
                                    3.7s
 ✔Container mexi-banco-s08-cuenta-3 Stopped
                                    3.8s
 ✔Container mexi-banco-s08-cuenta-1 Stopped
                                    3.8s
```

Aquí al final el origen quedo en $900 y el destino en $500

```
PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\sd-2027-1\entregas\lop
ez_miriam\s08\mexi-banco> Invoke-RestMethod -Uri "http://localhost:8081/api/
cuentas/1234567890123456" -Method Get

id clabe              titular           saldo
-- -----              -------           -----
 1 1234567890123456 Prueba S08 Origen 900.00


PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\sd-2027-1\entregas\lop
ez_miriam\s08\mexi-banco> Invoke-RestMethod -Uri "http://localhost:8081/api/
cuentas/9876543210987654" -Method Get

id clabe              titular            saldo
-- -----              -------            -----
 2 9876543210987654 Prueba S08 Destino 500.00


PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\sd-2027-1\entregas\lop
ez_miriam\s08\mexi-banco>
```

La falla dejó $100 descontados del origen sin que llegaran al destino. Esto pasó porque cargo y abono son peticiones HTTP independientes, y no existe una sola transacción de base de datos que deshaga ambas operaciones cuando una falla.

Parte 1: Saga orquestada

Modifiqué la entidad Transferencia para guardar un estado con un enum, ahora el servicio registra el progreso de la operación y cuando el abono falla intenta devolver el monto a la cuenta origen.

También agregué el movimiento COMPENSACION_TRANSFERENCIA a ambos servicios que manejan los tipos de movimiento.

El estado inicial se guarda desde el principio:

```
Transferencia t = transferencias.save(
new Transferencia(claveOrigen, claveDestino, monto));
```

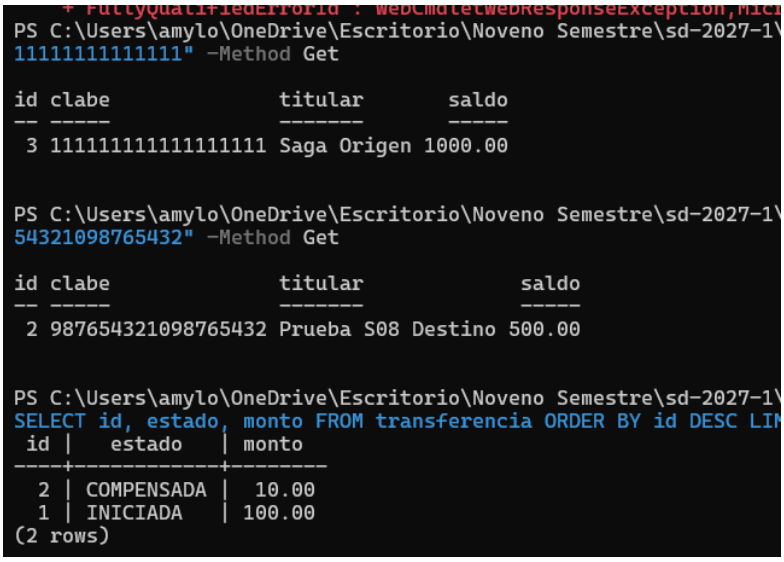
Al terminar el cargo se registra el avance:

```
CuentaRemota origen = cuentas.cargar(claveOrigen, monto);
t.setEstado(EstadoTransferencia.CARGADA);
transferencias.save(t);
```

Para probar una falla sin apagar todo el sistema, configuré una CLABE de destino en la variable CLABE_DESTINO_FALLO_ABONO. La simulación lanza una excepción antes de intentar abonar a ese destino. Dentro de la compensación, la idea principal es:

```
CuentaRemota devuelta = cuentas.abonar(claveOrigen, monto);
t.setEstado(EstadoTransferencia.COMPENSADA);
transferencias.save(t);
movimientos.registrar(devuelta.clabe(),
TipoMovimiento.COMPENSACION_TRANSFERENCIA,
monto, devuelta.saldo(), "Compensacion ...");
```

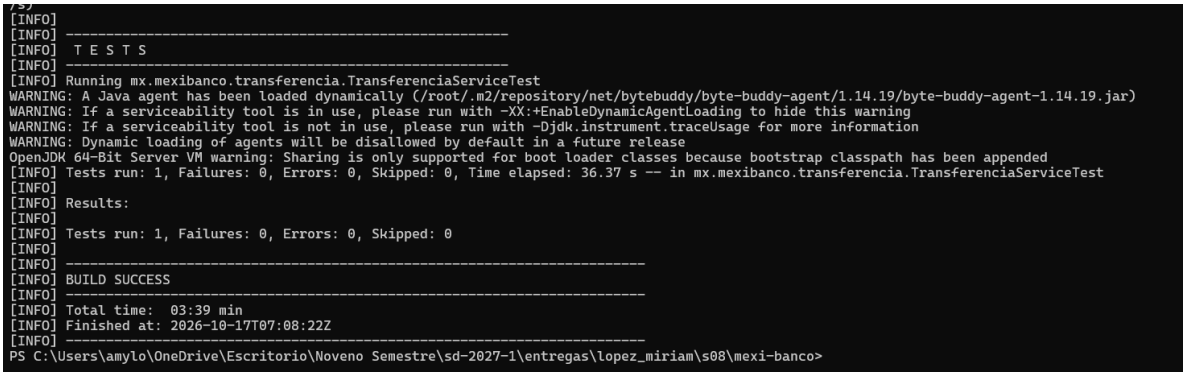
Después utilicé una cuenta nueva llamada Saga Origen, con $1,000, y una transferencia de $10 al destino de prueba con $500. La falla simulada se ejecutó antes del abono y la Saga devolvió el dinero. Al consultar de nuevo, el origen seguía con $1,000, el destino con $500 y la transferencia 2 aparecía como COMPENSADA.



Durante esta ejecución el Gateway devolvió 504 por una pausa que agotó su espera. El estado final y los saldos se comprobaron por separado, así que no tomé el HTTP 504 como prueba de que la devolución hubiera fallado.

Cuando también falla la compensación

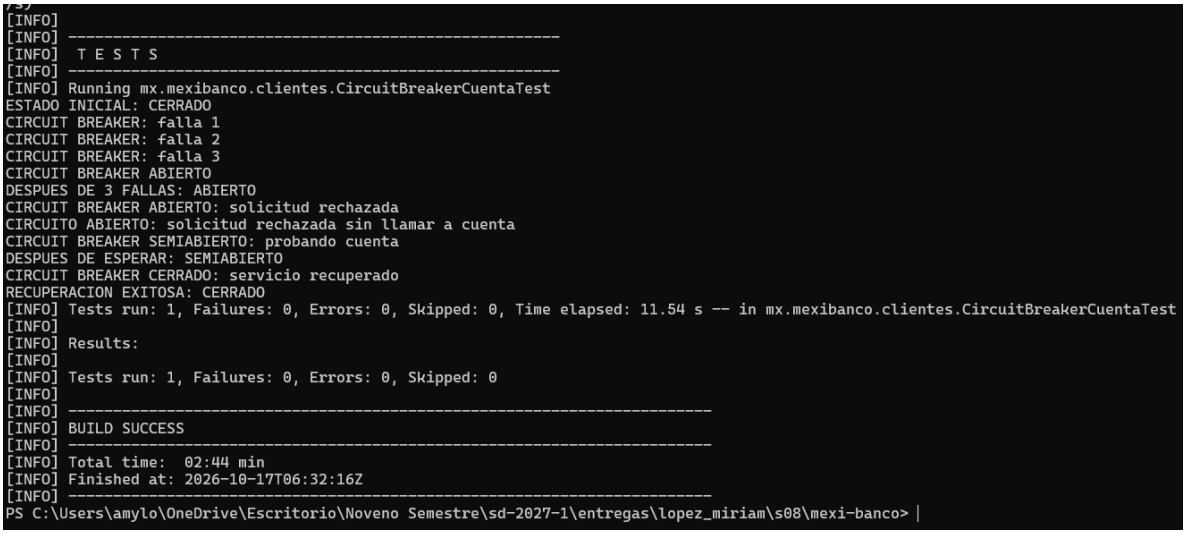
Agregué una prueba automatizada con Mockito que simula una falla del abono y después una falla al devolver el dinero. La prueba comprueba que la operación se mantiene en COMPENSACION_PENDIENTE. No hice una segunda prueba real apagando cuenta en este escenario, es por eso que la evidencia es una prueba simulada



Parte 2: Circuit Breaker

Implementé CircuitBreakerCuenta a mano, con estados CERRADO, ABIERTO y SEMIABIERTO. Lo conecté a consultar(), cargar() y abonar() de CuentaCliente. Después de tres errores de acceso a la red, abre el circuito por 10 segundos,mientras está abierto, rechaza la solicitud sin ejecutar la llamada, pasada la espera, permite una llamada de prueba; si funciona, regresa a CERRADO.

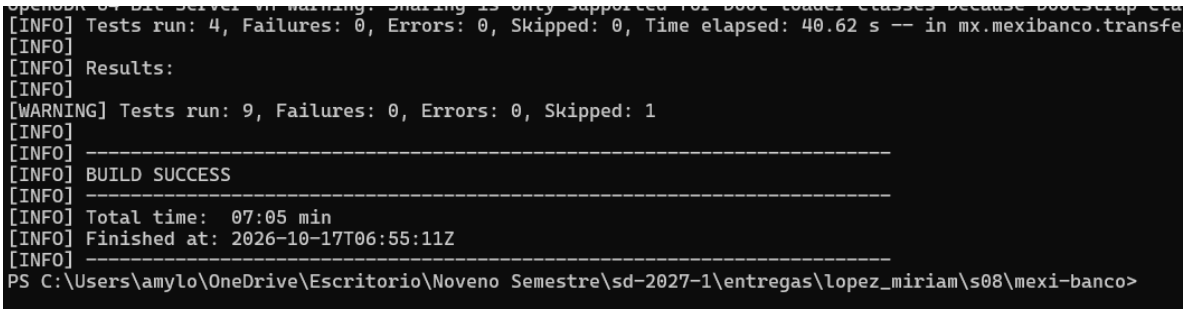
Comprobé las transiciones con una prueba unitaria que genera tres fallas simuladas, verifica el rechazo en ABIERTO y espera la recuperación para pasar de SEMIABIERTO a CERRADO.



La captura demuestra el comportamiento de la clase en una prueba controlada. No documenta una serie de transferencias contra una instancia real de cuenta caída.

Pruebas de Maven

Ejecuté las pruebas del servicio transferencia después de adaptar las pruebas antiguas al nuevo constructor y a los estados de la Saga. Maven terminó con BUILD SUCCES.



¿Qué no resuelve la Saga?

Aunque la Saga ayuda a devolver el dinero cuando una transferencia falla, no soluciona todos los problemas. Por ejemplo, si alguien revisa su saldo mientras se está haciendo una transferencia, puede ver menos dinero, aunque después se le devuelva.

También me di cuenta de que, cuando un servicio tarda mucho en responder, no siempre sabemos si el dinero se movió o no. Por eso todavía se necesitan otras formas de revisar las transferencias y evitar errores.

Conclusiones

En esta práctica aprendí que pueden ocurrir errores cuando los servicios se comunican entre sí. Primero hice una prueba donde se descontó dinero de una cuenta, pero no llegó a la otra.

Después implementé una Saga para intentar devolver el dinero cuando fallara una transferencia y un Circuit Breaker para controlar las fallas del servicio de cuentas.

Con esta práctica entendí que no basta con que el programa funcione, también es importante hacer pruebas y revisar que el dinero y los estados de las transferencias estén correctos.